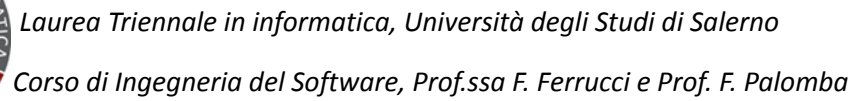




Design Pattern Document

C06 - CityClean

Riferimento	
Versione	0.5
Data	21/11/2025
Destinatario	Prof.ssa Filomena Ferrucci Prof. Fabio Palomba
Presentato da	C06 Team Nu Spettacl: ● Michele Bartoli (MB) ● Luigi Balsio (LB) ● Pio Cascone (PC) ● Domenico di Caprio (DC) ● Saverio Di Perna (SP) ● Antonio Pio Gargiulo (AG) ● Marco Ianniello (MI) ● Amanda Pia Caterina Robertazzi (AR) ● Vittorio Ruotolo (VR) ● Paolo Maria Siconolfi (PS) ● Davide Valiante (DV)
Approvato da	



Revision History

[illegible]



Team Members

Nome	Ruolo	Acronimo	Contatti
Antonio Ferrentino	Project Manager	AF	a.ferrentino50@studenti.unisa.it
Francesco Perilli	Project Manager	FP	f.perilli2@studenti.unisa.it
Michele Bartoli	Team Member	MB	m.bartoli@studenti.unisa.it
Luigi Balsio	Team Member	LB	l.balsio@studenti.unisa.it
Pio Cascone	Team Member	PC	p.cascone6@studenti.unisa.it
Domenico di Caprio	Team Member	DC	d.dicaprio2@studenti.unisa.it
Saverio Di Perna	Team Member	SP	s.diperna@studenti.unisa.it
Antonio Pio Gargiulo	Team Member	AG	a.gargiulo65@studenti.unisa.it
Marco Ianniello	Team Member	MI	m.ianniello28@studenti.unisa.it
Amanda Pia Caterina Robertazzi	Team Member	AR	a.robertazzi5@studenti.unisa.it
Vittorio Ruotolo	Team Member	VR	v.ruotolo4@studenti.unisa.it
Paolo Maria Siconolfi	Team Member	PS	p.siconolfi1@studenti.unisa.it
Davide Valiante	Team Member	DV	d.valiante@studenti.unisa.it



Laurea Triennale in informatica, Università degli Studi di Salerno

Corso di Ingegneria del Software, Prof.ssa F. Ferrucci e Prof. F. Palomba

Sommario

[Revision History](#)

[Team Members](#)

[1. Componenti Off-The-Shelf](#)

[2. Design Patterns](#)



1. Componenti Off-The-Shelf

Nell'implementazione del nostro sistema saranno utilizzati:

- Per l'implementazione lato Mobile (App) verrà utilizzato **Flutter**, il framework open-source specificato nell'SDD per lo sviluppo cross-platform (Android/iOS) del Presentation Tier.
- Per il modulo di Intelligenza Artificiale (come da requisito RF_U_14) verrà utilizzata **Ultralytics YOLOv11**, un'architettura di rete neurale per l'Object Detection.
- Per l'esecuzione del modello AI sui dispositivi mobili verrà utilizzato **ONNX Runtime**, motore di inferenza necessario per garantire i tempi di risposta inferiori ai 10 secondi (RNF_P_03).

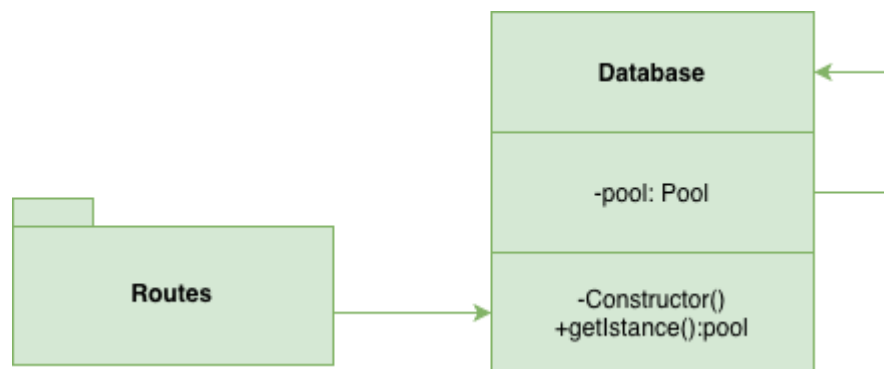
2. Design Patterns

Adapter

Il design pattern Adapter è un tipo di design pattern strutturale. Lo scopo principale di questo design pattern è facilitare la progettazione, semplificando le relazioni tra le entità. Infatti, permette ad oggetti con diverse interfacce di collaborare tra loro, attraverso la classe "adapter" che permette di convertire i dati in oggetti comprensibili dal sistema. All'interno del sistema l'Adapter è stato fondamentale per l'integrazione del modulo di Intelligenza Artificiale nell'applicazione mobile: esso converte il flusso di dati grezzo proveniente dalla fotocamera (frame in formato YUV/BGRA) in tensori numerici (Float32) comprensibili dal motore di inferenza YOLOv11, permettendo la comunicazione tra l'hardware del dispositivo e la logica di riconoscimento.

Singleton

Il design pattern Singleton è un tipo di design pattern di creazione. Lo scopo principale di questo tipo di design pattern è garantire che una classe abbia una singola istanza e fornire un punto di accesso globale a questa istanza da qualsiasi punto dell'applicazione. Il controllo dell'istanza viene permesso impostando il costruttore di default della classe privato e provvedendo a fornire il riferimento all'unica istanza di quella classe tramite un metodo. Nel caso specifico di CityClean, questo pattern verrà utilizzato in due contesti critici: lato Backend per gestire un'unica istanza del pool di connessioni al database PostgreSQL (ottimizzando le risorse), e lato Mobile per garantire che il pesante modello di Intelligenza Artificiale venga caricato in memoria una sola volta all'avvio e riutilizzato per ogni inferenza successiva.



DAO

Il design pattern DAO (Data Access Object) è un tipo di design pattern architetturale utilizzato principalmente per fornire un'interfaccia per l'archiviazione e l'accesso ai dati persistenti, senza la necessità di conoscere i dettagli implementativi per effettuare queste operazioni. Inoltre, è stato progettato per dividere la logica di business dalla logica di accesso ai dati: esso, infatti, fornisce metodi standardizzati finalizzati a nascondere i dettagli specifici del database consentendo cambiamenti nelle tecnologie. Nell'applicazione in questione, il DAO verrà implementato nel Backend Node.js per astrarre le query SQL dirette verso PostgreSQL. Provvederà a fornire quelle che sono le operazioni CRUD: Create, Read, Update e Delete (Creazione, Lettura, Aggiornamento ed Eliminazione) sulle entità Utente, Segnalazione e Rifiuto, mappandole ai rispettivi metodi HTTP.

Facade

Il design pattern Facade è un tipo di design pattern che offre ai vari sottosistemi delle interfacce per comunicare con gli altri sottosistemi. Tali interfacce evitano che un client debba conoscere come il sottosistema con cui si sta interfacciando funzioni, e quali siano le relazioni interne dei suoi vari componenti, solitamente complesse per loro natura. Le interfacce proposte dal pattern Facade offrono, al costo di un po' di efficienza, un modo ben determinato di chiedere servizi al sottosistema, lasciando ad esso la gestione interna dei propri componenti. Queste interfacce ci permettono di creare un'architettura chiusa, portando ad un debole accoppiamento tra il client e le componenti del sottosistema interessato.